

The Set That Says No

Transversal matroids in Lean 4: a tight-set proof of augmentation, and the canonical obstruction it makes possible

Carles Marín*

Independent researcher
karlesmarin@gmail.com

July 2026

Abstract

Mathlib has Hall’s marriage theorem. Mathlib has matroids. In the revision this work was checked against (84fdd0bf, 2026-07-15) nothing joins the two: every matroid in `Matroid/Constructions.lean` is trivial (`emptyOn`, `loopyOn`, `freeOn`, `uniqueBaseOn`), and none is built from combinatorial data. This paper closes that gap for the classical bridge between them, the *Transversal Matroid Theorem* of Edmonds–Fulkerson and, independently, Mirsky–Perfect: for a finite family $A = (A_0, \dots, A_{t-1})$ of finite sets, the partial transversals of A are exactly the independent sets of a matroid. I report a complete, `sorry`-free Lean 4 formalization over Mathlib of the finite-presentation form, packaged as `transversalMatroid A` with `transversalMatroid_indep` identifying its independent finite sets.

The contribution is not that Lean accepts the proof. It is the *shape* of the proof. Rather than certify an augmenting-path algorithm, the development runs Hall’s obstruction method and isolates one object in its own layer: a *tight* set R , one whose compatible slots are exactly saturated, $|N(R)| = |R|$. A failed insertion produces a tight witness; tight sets are closed under union; a union of witnesses forces $|J \setminus I| \leq |I \setminus J|$, contradicting $|I| < |J|$. Each step is a separate named lemma over Mathlib’s finite matching interface, and the separation pays a dividend the classical statement does not advertise. Since tight sets are closed under intersection as well as union, they form a *lattice*, so a partial transversal I carries a single largest tight subset `maxTight A I`, and

$$\text{insert } e \text{ } I \text{ is a partial transversal} \iff N(e) \not\subseteq N(\text{maxTight } A \text{ } I),$$

one canonical set deciding every insertion (`insert_iff_not_subset_maxTight`). Put in Mathlib’s own vocabulary, that same set computes the matroid’s closure operator on independent sets (`mem_closure_iff_subset_maxTight`): the library’s apparatus applying to the constructed object, rather than merely being said to apply. Six hundred and forty-five lines of Lean, thirty-four theorems, no `sorry`, `admit` or `native_decide`; `#print axioms` on all six headline theorems reports only `propext`, `Classical.choice`, `Quot.sound`. Section 2 situates the work against Hall, matching and matroid formalizations in Lean, Isabelle/HOL, Mizar and Coq, and Section 10 states exactly which priority claims are made, what was searched to support them, on what date, and what would refute them.

*Formalized with the assistance of an AI system (Claude, Anthropic); the mathematics is classical, and every claim, scope statement, and limitation is the author’s responsibility. What is *not* formalized is set out plainly in Section 9, and the Lean kernel, not the assistant, is what certifies the proofs.

What you will find here. The paper follows the development in the order it was built (Figure 1). Sections 1–2 say where the formal state of the art stops and what exactly is added. Section 3 replaces the search for a matching by a counting inequality, Section 4 turns the *failure* of that inequality into an object and proves augmentation from it, and Section 5 explains why that route, split into three layers, is the contribution rather than a stylistic preference. Section 6 packages the matroid; Section 7 collects the dividend the architecture pays. The load-bearing lemmas are in Section 8, the honest limits in Section 9, the conclusion in Section 11. If you read only two things, read the contribution box at the end of Section 1 and Theorem 1.

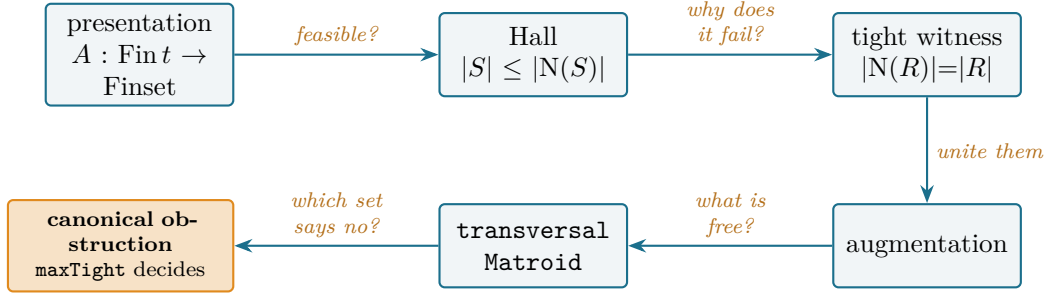


Figure 1: The reader’s map. Every node is a block of **sorry-free** Lean and every arrow is the question its section answers. The chain runs left to right and back: from a finite presentation to Hall’s inequality, from a *failure* of that inequality to a tight set, from many tight sets to one, and from the matroid to the single canonical set that decides every insertion.

1 Two libraries that do not meet

Tasks must be assigned to distinct qualified agents. A task may have several compatible agents, two tasks cannot share one agent, and the question is which collections of tasks can be served *simultaneously*. The compatibility relation is local and innocuous; the feasibility condition is neither, because admitting one new task may force the reassignment of several old ones. This is the terrain of systems of distinct representatives and of Hall’s marriage theorem [1].

Write the presenting family as

$$A = (A_0, \dots, A_{t-1}), \quad A_i \subseteq \alpha,$$

call x *compatible* with the index i when $x \in A_i$, and call a finite set $I \subseteq \alpha$ a *partial transversal* when its elements can be assigned injectively to compatible indices. The empty set is feasible; a subset of a feasible set is feasible, by restricting the assignment. Neither observation reaches the fact that matters:

if I and J are partial transversals and $|I| < |J|$, then some $e \in J \setminus I$ makes $I \cup \{e\}$ a partial transversal.

That is the matroid exchange axiom, and it is not visible in the definition. Its history is worth a paragraph, because this paper repeats it.

Two papers from 1935. In the *Journal of the London Mathematical Society*, Philip Hall gave the exact condition for a family of sets to admit a system of distinct representatives [1]. In the *American Journal of Mathematics*, Hassler Whitney abstracted the properties of linear dependence and named the result a *matroid* [10]. They shared a year and little else: one asked when a set

of choices can be made, the other axiomatised what it means for choices to be independent, and neither refers to the other.

They stayed apart for a generation. Whitney’s paper drew a short burst of interest and then matroid theory went quiet for over twenty years, until Tutte’s papers of 1958–59 characterised the regular and graphic matroids [11]; Rado had meanwhile pursued the same ideas in the 1940s, proving the matroid generalisation of Hall’s criterion [6]. The revival came in the 1960s, and with it two places where the two 1935 papers were finally read together. Jack Edmonds convened the first conference on matroid theory at the National Bureau of Standards in 1965 [9]; and at Sheffield, Leon Mirsky, Hazel Perfect and John Pym were building transversal theory out of Hall’s theorem and its generalisations. Both groups arrived at the same bridge. The *Journal of Research of the NBS* carried Edmonds and Fulkerson’s version in the conference year [2]; Mirsky and Perfect’s, which also covers infinite families, appeared in 1967 [3]. Thirty years after 1935, the marriage theorem and the axioms of independence turned out to be about one object.

Perfect kept going in that direction. In 1968 she applied Menger’s theorem to produce the class of matroids now called *gammoids* [4]—a class that contains the transversal matroids [17, 18], so that the object formalized in this paper is a special case of one she introduced. Her doctorate followed the year after, in 1969, under the title *Studies in Transversal Theory with Particular Reference to Independence Structures and Graphs* [7]: the thesis came after the theorem. She was a Reader at Sheffield, wrote *Independence Theory in Combinatorics* with Victor Bryant in 1980 [12], and the conjecture she made with Mirsky in 1965 on which points of the unit disc are eigenvalues of $n \times n$ doubly stochastic matrices [8]—proved by them for $n \leq 3$, confirmed for $n = 4$, refuted for $n = 5$ —is open beyond that. Hers is the name that puts this result on *Theorem of the Day*’s list of theorems by women mathematicians [5], from which the target was drawn; the attribution used throughout this paper is the one that page prints.

Why it matters practically is the greedy corollary. To say partial transversals form a matroid is exactly to say that a cheapest maximal transversal can be selected greedily—cheapest agents first, assignment settled afterwards—which is false for the naive matching search, where a greedy prefix may have to be abandoned [5]. The *Theorem of the Day* page ends on a caveat that this paper takes seriously and answers in Section 7: greediness “leaves open the question of how we know our partial transversals are valid unless we find a matching at the same time.”

Where formal libraries stand—which is to say, in 1935. The situation this paper addresses is the pre-1965 one, reproduced inside a proof assistant. Mathlib [26], the mathematical library of Lean 4 [27], has Hall’s marriage theorem, contributed by Gusakov, Mehta and Miller [19], in the strong form we need: a finite matching interface (`hallMatchingsOn`) and the equivalence `Finset.all_card_le_biUnion_card_iff_existsInjective`.

Mathlib also has a substantial matroid development, due largely to Peter Nelson, with bases, circuits, closure, duality, minors and rank. What it does not have is a bridge. Every matroid in `Mathlib/Combinatorics/Matroid/Constructions.lean` is trivial—`emptyOn`, `loopyOn`, `freeOn`, `uniqueBaseOn`—and none of them is built from combinatorial data. Searching revision `84fdd0bf` for *transversal* returns the group-theoretic sense (cosets, Schreier, Schur–Zassenhaus) and one prose mention inside the Hall module; there is no transversal matroid. The two libraries sit beside each other and do not meet.

What this paper adds

Existing formalizations of matching and of Hall’s theorem do not, by themselves, yield a transversal matroid: they certify *that* a matching exists, and the matroid axiom needs a reason *why* an insertion fails. The contribution of this paper is threefold.

1. **The construction.** `transversalMatroid A`, with its independent finite sets identified exactly (`transversalMatroid_indep`), plugged into Mathlib’s matroid API through `IndepMatroid.ofFinset`. To my knowledge, and against the searches dated and itemised in Section 10, it is the first matroid in Mathlib built from a combinatorial presentation rather than by a trivial or dual construction—a statement about what I searched, not a proof of absence.
2. **The proof architecture, which is the point.** Augmentation is proved by a Hall-obstruction argument organized around *tight* sets, split into three independent layers—matching, tight-set, counting—each a separately named lemma. No augmenting-path algorithm is formalized, and no monolithic tactic block appears. The tight-set layer is library-shaped: it is about set families and Hall’s inequality, not about matroids.
3. **The dividend that architecture pays.** Because the obstruction was isolated, one can ask what tight sets do as a family. They are closed under intersection as well as union, hence form a lattice; a partial transversal therefore has a *maximum* tight subset `maxTight A I`, and a single containment test against it decides every insertion (Theorem 1). That is not visible from the classical statement of the theorem, and it is what the layered proof makes cheap.

2 Related work

The point of this section is orientation, not priority. Three literatures converge on the present target—formal Hall theorems, formal matching theory, and formal matroid theory—and the transversal matroid sits precisely where none of them ends up.

Hall’s marriage theorem, formalized. Hall’s 1935 criterion [1] is a standard target for proof assistants. In Isabelle/HOL, Jiang and Nipkow contributed *Hall’s Marriage Theorem* to the Archive of Formal Proofs in 2010 [20], with both the finite and the countable-index versions. In Lean, Gusakov, Mehta and Miller formalized it in Mathlib [19], in three interchangeable phrasings (indexed families of `Finsets`, a relation on finite types, and a `Fintype` filter form), together with the compactness step of Halpern that lifts the finite theorem to arbitrary index types. Neighbouring finite-set theorems have been mechanized elsewhere—Dilworth’s and Mirsky’s theorems in Coq [22], and Dilworth’s again in the AFP (2025)—so the marriage theorem sits inside the settled part of the formal combinatorics landscape rather than at its edge. *What this gives and does not give.* These developments certify the existence of a system of distinct representatives under the Hall inequalities. They do not say what happens when the inequalities fail, and the matroid axiom is entirely a statement about failure.

Matching theory, formalized. Beyond the existence theorem, formal matching theory has concentrated on algorithms and on the deeper structure theorems: Berge’s augmenting-path criterion, blossom-based maximum matching, and the algorithmic corner of combinatorial optimization, chiefly

in Isabelle/HOL. The most directly relevant entry is the ITP 2025 development *A Formal Analysis of Algorithms for Matroids and Greedoids* [23], which includes executable bipartite maximum matching and verified greedy algorithms. This is the piece of prior art closest to the present target and the reason no algorithmic claim is made here: that literature already owns the algorithmic side. *What this gives and does not give.* An executable maximum-matching routine decides feasibility, and one can recover an obstruction by inspecting a failed search. But the certificate is then an artifact of a particular algorithm run, not an object with a theory; nothing in that route tells you that obstructions form a lattice.

Matroids, formalized. Isabelle/HOL has Keinholtz’s AFP entry *Matroids* (2018) [21], developing independent sets, bases, circuits and rank, and the ITP 2025 matroid/greedoid algorithm work above. In Lean 4 the largest matroid development is Peter Nelson’s, which follows the infinite-matroid axioms and whose core has been upstreamed into Mathlib; recent work in the same ecosystem includes a formalization of the excluded-minor characterization of binary matroids [24] and a verified composition direction of Seymour’s theorem for regular matroids [25]. The Lean matroid area is active and structural. *What this gives and does not give.* These libraries are rich in what one can prove *about* a matroid and poor in matroids to prove it about. Concretely, in revision 84fdd0bf (2026-07-15) the constructions module offers only the empty, loopy, free and unique-base matroids. Graphic matroids, linear matroids and transversal matroids—the three classical families that make matroid theory a theory of examples—are, in that revision, absent.

Transversal theory, formalized. In searches carried out in July 2026 and itemised in Section 10, I located no formalization of the Transversal Matroid Theorem: not in Lean/Mathlib (a text search of revision 84fdd0bf returns only the group-theoretic sense of *transversal*), not in the Isabelle AFP, not in Mizar, not in the Coq `mathcomp` ecosystem, and not in HOL Light. This is a negative search result and is recorded as such in Section 9: it is an audit status, not a novelty claim, and the neighbouring AFP matroid/greedoid algorithm work is close enough that any claim of first formalization of the surrounding *area* would be false.

The gap, stated exactly. Formal Hall theorems certify feasibility; formal matching work certifies algorithms; formal matroid work certifies structure over an empty stock of examples. The transversal matroid needs all three at once, and the join is not mechanical: the matroid axiom is a statement about *simultaneous* failure of many insertions, and no existence theorem, however strong, delivers it. Supplying that join—with an obstruction object factored out so that it can be reused and asked questions of—is what this paper does.

3 Feasibility as a counting inequality

For an element x , the Lean development names its compatible indices `compatibleSlots A x : Finset (Fin t)`, and for a finite S writes

$$N(S) = \bigcup_{x \in S} \text{compatibleSlots } A x \quad (\text{slotsOf } A S).$$

`IsPartialTransversal A I` is defined as the nonemptiness of Mathlib’s `hallMatchingsOn (compatibleSlots A) I`: an injective function on the subtype of elements of I , landing in the compatible slots. Putting the object in the library’s own language, rather than in a bespoke definition, is what makes the rest of the development short.

The first substantive result is the exact finite equivalence

$$I \text{ is a partial transversal} \iff |S| \leq |N(S)| \text{ for every } S \subseteq I.$$

The forward direction restricts a matching to S and injects S into $N(S)$; the reverse delegates the construction of representatives to Mathlib’s finite Hall theorem on the subtype of elements of I . The asymmetry is deliberate proof engineering: one direction is a cardinal injection we can do by hand, the other is exactly the library’s job. In Lean the two halves are `partialTransversal_hall` and `hall_partialTransversal`, paired as `partialTransversal_iff_hall`.

- tight block $R = \{a, b, c\}$ and its three slots
- d : slack, outside the obstruction
- x refused: $N(x)$ lies inside $N(R)$
- y admitted: $N(y)$ escapes $N(R)$

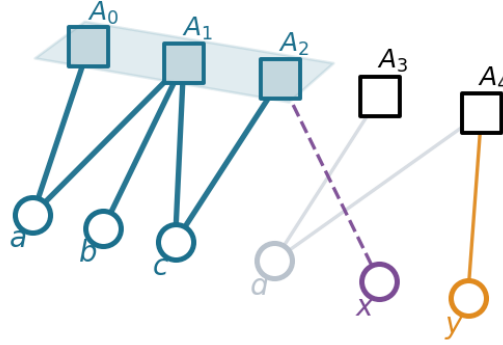


Figure 2: The running example, drawn by `generate_obstruction_figures.py` from the same data that `check_canonical_obstruction.sage` verifies. Five presenting sets over $\{a, b, c, d, x, y\}$ give compatible slots $N(a) = \{0, 1\}$, $N(b) = \{1\}$, $N(c) = \{1, 2\}$, $N(d) = \{3, 4\}$, $N(x) = \{2\}$, $N(y) = \{4\}$. The set $I = \{a, b, c, d\}$ is a partial transversal ($a \mapsto 0$, $b \mapsto 1$, $c \mapsto 2$, $d \mapsto 3$). The shaded plane is the saturated neighbourhood of the tight block $R = \{a, b, c\}$: three elements, three slots, nothing to spare. It is the wall that refuses x and lets y through.

Read on the example of Figure 2: for $I = \{a, b, c, d\}$ every subset has at least as many slots as elements, so I is feasible. For `insert  $x$   $I$` the subset $\{a, b, c, x\}$ has four elements and only the three slots $\{0, 1, 2\}$; Hall fails at once. What matters next is that this failure has a *shape*. The question handed forward is: when every attempted insertion fails, what do those failures share?

4 Why a larger transversal must donate an element

Let I, J be partial transversals with $|I| < |J|$, and suppose for contradiction that no $e \in J \setminus I$ can be inserted into I .

For each such e , the Hall characterization gives $S \subseteq \text{insert } e \text{ } I$ with $|N(S)| < |S|$. Since I itself satisfies Hall, S must contain e (otherwise S would be a violating subset of I); this is `hall_violation_insert_mem`. Remove e , call the remainder R , and a short cardinality squeeze upgrades the strict inequality into four exact facts:

$$R \subseteq I, \quad e \notin R, \quad |N(R)| = |R|, \quad N(e) \subseteq N(R).$$

The tight witness (recipe)

Call R **tight** when it exactly saturates its compatible slots, $|N(R)| = |R|$ (**IsTight**): it has no slack at all. A tight $R \subseteq I$ with $N(e) \subseteq N(R)$ is a *certificate that e has nowhere new to go*—every slot e could use is already spoken for by elements of R , and R has no room to shuffle. In Lean the extraction is **exists_tight_witness_of_not_insert**: a failed insertion is never an unexplained search failure; it always hands back this object.

On the running example with $I = \{a, b, c, d\}$, the attempt to add x returns the tight witness $R = \{a, b, c\}$: three elements filling the three slots $\{0, 1, 2\}$, and $N(x) = \{2\}$ lies inside them (Figure 3).

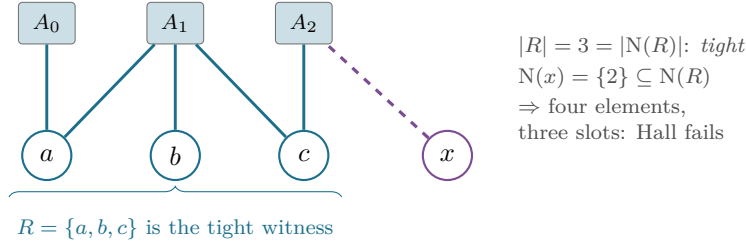


Figure 3: A failed insertion, explained rather than merely observed. The three shaded slots are saturated by $\{a, b, c\}$, and every slot x could occupy already lies among them. The certificate is a set, not a trace of a failed search.

One tight set is local. Augmentation needs a *single* obstruction covering all of $J \setminus I$ at once, and the pivotal fact is that tightness survives union. If $R, S \subseteq I$ are tight then, because $N(R \cup S) = N(R) \cup N(S)$ and $N(R \cap S) \subseteq N(R) \cap N(S)$,

$$|N(R \cup S)| + |N(R \cap S)| \leq |N(R)| + |N(S)| = |R| + |S| = |R \cup S| + |R \cap S|,$$

while Hall bounds each left-hand term from below by its right-hand partner. Both inequalities are therefore forced to equalities, and $R \cup S$ is tight (**tight_union**); induction gives the finite-family version **tight_biUnion**.

Now choose a tight witness R_e for each $e \in J \setminus I$, set $U = \bigcup_e R_e$, and apply Hall to $P = (J \setminus I) \cup (U \cap J) \subseteq J$. Its neighbourhood lies inside $N(U)$, and U is tight, so cardinal arithmetic yields $|J \setminus I| \leq |I \setminus J|$ (**card_sdiff_le_of_tight_cover**). But for finite sets $|I| < |J|$ is equivalent to $|I \setminus J| < |J \setminus I|$. Contradiction, and augmentation follows:

```
theorem partialTransversal_augmentation
  (hI : IsPartialTransversal A I) (hJ : IsPartialTransversal A J)
  (hcard : I.card < J.card) :
  ∃ e ∈ J \ I, IsPartialTransversal A (insert e I)
```

This is a Hall/tight-set proof of augmentation. It does not implement maximum matching or augmenting-path search: an existence theorem and a certified algorithm are different deliverables, and only the first is claimed here.

5 The three layers of the proof

The difficulty is not the empty set, not restriction, and not Hall's theorem, which Mathlib already supplies. The genuine proof engineering is the route from *several* failed insertions to *one* global

contradiction. Written as a single argument it is a paragraph; written as a single tactic block it is unreadable and un reusable. The development therefore separates it into three layers, and this separation is the paper’s central technical claim.

The three layers

Matching layer. `compatibleSlots`, `IsPartialTransversal`, `restriction`, and `partialTransversal_iff_hall`: a stable interface between injective assignments and cardinal inequalities. Everything above it speaks only of cardinalities.

Tight-set layer. `exists_tight_witness_of_not_insert` turns a failure into an object; `tight_union` and `tight_biUnion` give that object an algebra. This layer mentions no matroid and no matching—only set families and Hall’s inequality—which is exactly why it is reusable, and why Section 7 costs almost nothing.

Counting layer. `card_sdiff_le_of_tight_cover` takes a finite family of tight subsets covering the slots of $J \setminus I$ and returns the single inequality $|J \setminus I| \leq |I \setminus J|$. The augmentation theorem then merely chooses one witness per element of the finite subtype $J \setminus I$ and invokes it.

This is the formal analogue of drawing the obstruction before counting it. In Lean the details are concrete: witnesses are indexed by a finite subtype and chosen with `Exists.choose`; their union is a `Finset.biUnion`; a neighbourhood is itself a `biUnion`; the final contradiction is `Finset union/intersection/difference arithmetic` discharged by `omega`. None is deep in isolation. Keeping them in separately named lemmas is what stops a short combinatorial paragraph from becoming one opaque proof, and it is what makes the tight-set layer available to anyone who wants it for a different purpose.

The claim is worth stating without hedging: *the novelty is not that Lean accepts the proof. It is that the proof architecture isolates the tight-set obstruction into reusable lemmas.* That is simultaneously a mathematical choice—obstruction method rather than augmenting paths—and a proof-engineering one, and Section 7 is the evidence that the choice was the right one.

6 Packaging the matroid

Mathlib’s `IndepMatroid.ofFinset` turns a finite-set independence predicate satisfying the empty-set, hereditary and augmentation axioms into a finitary matroid. The development applies it with ground set `Set.univ` and predicate `IsPartialTransversal A`:

```
noncomputable def transversalMatroid (A : Fin t → Finset α) : Matroid α
```

The universal ground set is deliberate. An element of α occurring in no presenting set is simply a *loop*: no partial transversal contains it. The final theorem is therefore exact without first restricting α to $\bigcup_i A_i$:

```
theorem transversalMatroid_indep (A : Fin t → Finset α) (I : Finset α) :
  (transversalMatroid A).Indep (I : Set α) ↔ IsPartialTransversal A I
```

This is the finite-presentation Transversal Matroid Theorem, as an object that downstream Mathlib matroid results—rank, closure, bases, duality, minors—can consume directly; Corollary 1 exercises the closure operator on it, so the claim is demonstrated rather than asserted. The distinction between an *available* theorem and an *implemented* application is kept: the construction makes the interface available; it does not, here, implement a weighted greedy optimizer, a rank oracle, or any complexity analysis.

7 The obstruction is canonical

Section 4 needed only that tight sets are closed under *union*. Ask the tight-set layer one more question—what about intersection?—and the same submodular squeeze answers it, because the displayed chain of inequalities forces *both* sides to equality at once. In Lean this is a single lemma returning a conjunction, `tight_union_and_inter`; the intersection half, `tight_inter`, is the half augmentation never uses.

Proposition 1 (The tight sets are a lattice; `tight_union_and_inter`). *Let I be a partial transversal and $R, S \subseteq I$ tight. Then $R \cup S$ and $R \cap S$ are tight. Hence the tight subsets of I form a sublattice of the Boolean lattice on I , containing $\emptyset$.*

That the equality cases of Hall’s condition are closed under union and intersection is classical matching theory—it is the submodularity of N , the engine behind Ore’s defect formula [13] and the Dulmage–Mendelsohn decomposition [14], and it is standard in Lovász and Plummer [15]. Nothing new is claimed for the mathematics. What is new is that the layered development gets it for the price of one extra conjunct, and that the consequence below can then be stated at all.

Since the family is finite and closed under union, it has a maximum. The development defines

$$\text{tightSubsets } A I = \{R \in \mathcal{P}(I) : R \text{ tight}\}, \quad \text{maxTight } A I = \bigcup \text{tightSubsets } A I,$$

and proves that this union is itself tight (`maxTight_isTight`, via `tight_biUnion`) and contains every tight subset (`subset_maxTight`). It is the *maximum*, not merely a maximal element.

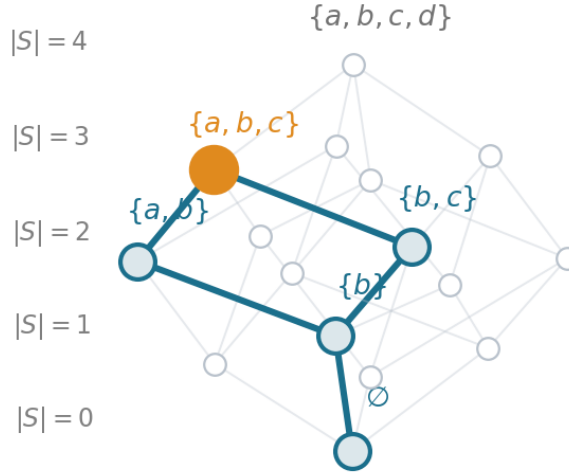
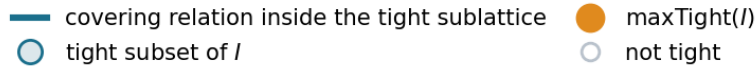


Figure 4: The lattice of tight subsets of $I = \{a, b, c, d\}$, drawn inside the Boolean lattice on I (height $= |S|$); `maxTight(I)` tops it, and I itself is not tight. Only five of the sixteen subsets are tight, and they form a genuine lattice, not a chain: $\{a, b\} \cap \{b, c\} = \{b\}$ and $\{a, b\} \cup \{b, c\} = \{a, b, c\}$ are both tight (Proposition 1). The top, in amber, is `maxTight(I) = {a, b, c}`—a *proper* subset of I , since I itself has four elements and five slots. One containment test against its neighbourhood decides every insertion.

The reward is a decision statement. The existence lemma of Section 4 says a failed insertion *produces* a tight cover; its converse is elementary—if $R \subseteq I$ is tight, $e \notin R$ and $N(e) \subseteq N(R)$, then `insert e R` has $|R| + 1$ elements and only $|R|$ slots, so Hall fails (`not_insert_of_tight_cover`). Necessary and sufficient together, and then monotonicity of N collapses the existential onto the maximum:

Theorem 1 (One set decides every insertion; `insert_iff_not_subset_maxTight`). *Let I be a partial transversal of A and $e \notin I$. Then*

$$\text{insert } e I \text{ is a partial transversal} \iff N(e) \not\subseteq N(\text{maxTight } A I).$$

The criterion is stated above in the development’s own vocabulary. It can be said in Mathlib’s, and that is the form in which it becomes a statement about a matroid rather than about a bespoke predicate:

Corollary 1 (The obstruction computes the closure; `mem_closure_iff_subset_maxTight`). *With I and e as above,*

$$e \in (\text{transversalMatroid } A).\text{closure } I \iff N(e) \subseteq N(\text{maxTight } A I).$$

`Matroid.closure` is Mathlib’s own operator, carrying its own theory of flats, spanning sets and rank; the corollary says that on independent sets that operator is decided by one containment test against one canonical finite set—no search over subsets, no matching recomputed. The proof is three lines: Mathlib’s `Indep.notMem_closure_iff_of_notMem` turns membership in the closure into failure of an insertion, `transversalMatroid_indep` turns that into a statement about partial transversals, and Theorem 1 finishes. This is also the promise of Section 6 discharged: the library’s apparatus does apply to the constructed object, and here it is applying.

This is the answer to the caveat *Theorem of the Day* leaves open—how do we know our partial transversals are valid without finding a matching at the same time? For a fixed I we do not need to: one tight set, computed once, answers every subsequent membership question by a containment test. In the running example `maxTight` = $\{a, b, c\}$ with $N = \{0, 1, 2\}$, so $N(x) = \{2\}$ is inside and x is refused, while $N(y) = \{4\}$ is outside and y is admitted (Figure 4). An exhaustive cross-check over that presentation—all 56 partial transversals and all 180 insertion decisions—agrees with the theorem in every case.

Two honest qualifications. First, `maxTight` is defined by a filter over the powerset of I , so the *definition* is exponential; the theorem certifies the characterization, not an efficient procedure, and computing the maximum tight set efficiently (it is a Dulmage–Mendelsohn question) is left to the algorithmic literature. Second, as with Proposition 1, the mathematics is classical; the contribution is that it is formal, and that it fell out of the architecture rather than requiring a new development.

8 The building blocks

Table 1 gathers the load-bearing results; Table 2 measures the development. The dependency chain is short: the matching layer gives the Hall equivalence; the Hall equivalence gives the tight witness; `tight_union_and_inter` gives the tight sets an algebra; the counting layer turns a cover into one inequality; and the lattice gives the canonical obstruction.

all 56 partial transversals of the presentation; 180 insertion decisions,
every one of them predicted by $N(e) \subseteq N(\maxTight(I))$

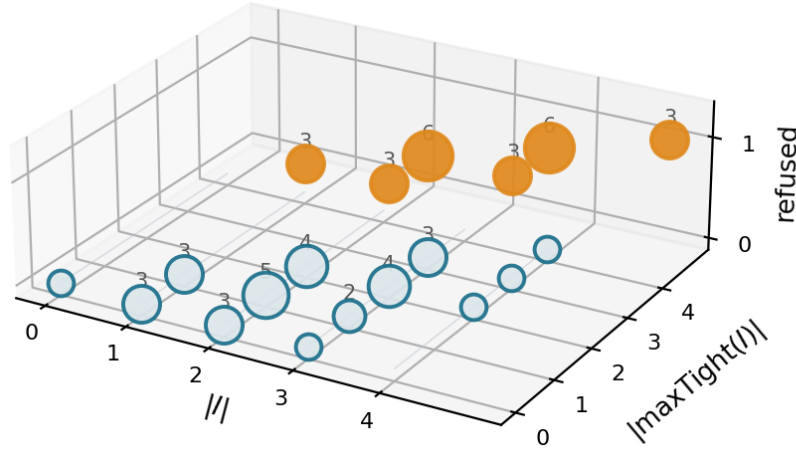


Figure 5: The criterion, exercised over the whole presentation rather than one example. Every partial transversal I of the running family is plotted at $(|I|, |\maxTight(I)|, \#\text{refused})$, where a refusal means an element outside I that cannot be inserted; marker area counts how many I share a position. Amber marks the positions where the canonical obstruction actually bites. All 56 partial transversals and all 180 insertion decisions agree with Theorem 1; the plot is the exhaustive check of `check_canonical_obstruction.sage`, not a sample.

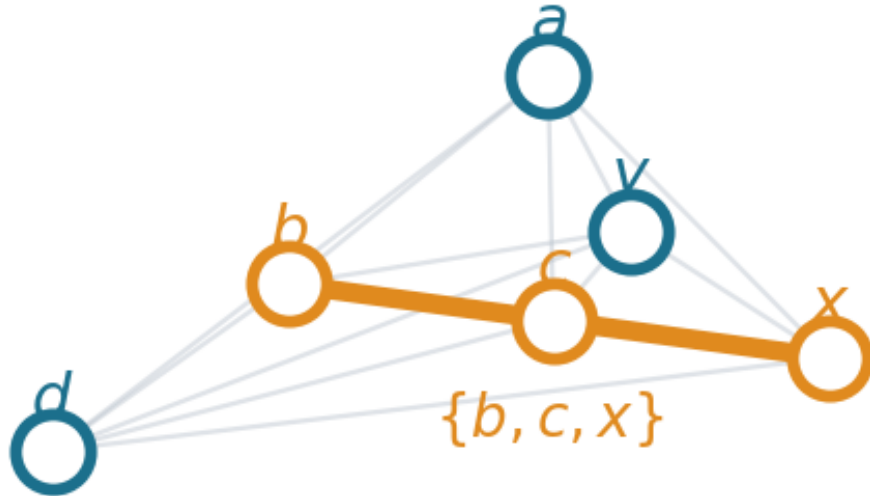


Figure 6: The same presentation seen as the matroid it defines: rank 5 on six elements, so exactly one circuit. Twelve of the thirteen rank-2 flats carry two points; one carries three, and the drawing honours that by placing b , c and x on a line. That line *is* the Hall violator, $N(\{b, c, x\}) = \{A_1, A_2\}$ —three points competing for two slots—and every refusal anywhere in this presentation traces back to it. The circuit and the tight obstruction are the same phenomenon read from opposite sides: a tight set is what remains of a circuit after one of its elements is removed.

Lean name	Statement	Status
<i>TransversalMatroid.lean</i> — the construction		
<code>partialTransversal_empty</code>	$\emptyset$ is a partial transversal	proved
<code>partialTransversal_subset</code>	restriction of a matching (heredity)	proved
<code>partialTransversal_iff_hall</code>	feasible $\iff S \leq N(S) $ on all $S \subseteq I$	proved
<code>hall_violation_insert_mem</code>	a violating subset must contain e	proved
<code>exists_tight_witness_of_not_insert</code>	failed insertion $\Rightarrow$ tight witness	proved
<code>tight_union, tight_biUnion</code>	tightness survives finite union	proved
<code>card_sdiff_le_of_tight_cover</code>	$ J \setminus I \leq I \setminus J $ from a cover	proved
<code>partialTransversal_augmentation</code>	the exchange axiom	proved
<code>transversalMatroid_indep</code>	Indep $\iff$ partial transversal	proved
<i>TransversalObstruction.lean</i> — the canonical obstruction		
<code>tight_union_and_inter</code>	tight sets form a lattice	proved
<code>maxTight_isTight</code>	the top of the lattice is tight	proved
<code>subset_maxTight</code>	it is the maximum, not just maximal	proved
<code>not_insert_of_tight_cover</code>	a tight cover blocks the insertion	proved
<code>not_insert_iff_exists_tight</code>	failure $\iff$ some tight cover	proved
<code>insert_iff_not_subset_maxTight</code>	one set decides every insertion	proved
<code>mem_closure_iff_subset_maxTight</code>	it computes <code>Matroid.closure</code>	proved

Table 1: The development, all `sorry-free`, Lean 4 / Mathlib. `#print` axioms on `partialTransversal_augmentation`, `transversalMatroid_indep`, `tight_union_and_inter`, `maxTight_isTight`, `insert_iff_not_subset_maxTight` and `mem_closure_iff_subset_maxTight` reports only `propext`, `Classical.choice`, `Quot.sound`.

Quantity	Value
Lines: construction / obstruction	400 / 245
Theorems + definitions	15+4 / 19+5
Direct Mathlib imports	3: <code>Combinatorics.Hall.Basic</code> , <code>Matroid.IndepAxioms</code> , <code>Matroid.Closure</code>
<code>sorry</code> / <code>admit</code> / <code>native_decide</code>	0
Axioms beyond Lean’s core	0: only <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
Build	lake build <code>Theorems.TransversalObstruction</code> — 1164 jobs, exit 0
Compile, warm Mathlib	≈ 4.7 s (obstruction module)
Toolchain	Lean 4 v4.32.0; Mathlib 84fdd0bf (2026-07-15)
Sage cross-check	56 partial transversals, 180 insertion decisions

Table 2: The development measured. The footprint is deliberately light: two Mathlib imports, no heavy algebra, because the argument is elementary counting once the layers are separated. The Sage [28] row is an executable cross-check of the running example, not evidence for the theorems; the Lean kernel is that evidence.

9 The boundary of the contribution

What is proved. The Transversal Matroid Theorem for finite presentations $A : \text{Fin } t \rightarrow \text{Finset } \alpha$: partial transversals satisfy the three independent-set axioms and are exactly the independent finite sets of `transversalMatroid A`; plus the tight-set lattice, the maximum tight subset, and the insertion criterion of Theorem 1. All `sorry-free`, with the axiom profile above.

Finite presentations only. The family is finite and each A_i is finite. The ambient type α need not be finite, but the development characterizes the *finite* independent sets and uses Mathlib’s finitary construction. The infinite version—the one Mirsky and Perfect also proved [3]—is not covered.

No algorithmic claim. The proof selects tight witnesses with classical choice. It proves augmentation; it does not implement augmenting-path search, maximum matching, or any bound. `maxTight` is defined by a powerset filter and is not an efficient procedure.

The mathematics is classical. Edmonds–Fulkerson and Mirsky–Perfect proved the theorem; the tight-set method is standard transversal and matching theory [13, 14, 15, 16, 17]. What is offered here is the formalization, its architecture, and the packaging of the obstruction as a canonical object.

Authorship. The formalization was carried out in close interaction with an AI assistant (Claude, Anthropic): the choice of target, the proof architecture, the statements, and the scope and novelty claims are mine; the assistant drafted and iterated much of the Lean under that direction. This does not bear on validity—a proof is accepted only when the Lean kernel type-checks it, which it does, with the axiom profile reported above—but it is stated plainly for attribution.

Trust boundary. The three reported axioms describe the theorems as checked by the current Lean environment. They are not a claim to have eliminated the trusted Lean kernel, compiler, or hardware.

10 Priority: what was searched, when, and what would refute it

Claims of the form “this had not been formalized” decay. They are true of a moment, a set of libraries and a set of queries, and every one of those changes. This section fixes all three so that a reader in a later year can check the claim rather than trust it, and—if it has since become false—can say exactly when it stopped being true.

What is claimed. Two things, and nothing beyond them. (i) To my knowledge, the matroid `transversalMatroid` is the first in Mathlib constructed from a combinatorial presentation rather than by a trivial or dual construction. (ii) To my knowledge, the Transversal Matroid Theorem had not been formalized in an interactive theorem prover before this development.

Both are claims about *formalization*, never about the mathematics, which is Edmonds–Fulkerson’s and Mirsky–Perfect’s and sixty years old.

What is not claimed. No priority for any mathematical statement in this paper. No claim that the surrounding area is unformalized—it is not: the AFP carries verified algorithms for matroids and greedoids including bipartite maximum matching [23], and Mathlib carries a substantial structural matroid theory. No claim about work I could not read, including unpublished developments, private repositories, and formalizations filed under a different name.

Exactly what was searched, and when. All of the following in July 2026.

- **Mathlib**, revision 84fdd0bf (2026-07-15), toolchain v4.32.0: a case-insensitive text search for *transversal* over the whole of Mathlib/, which returns `Combinatorics/Hall/Basic.lean` (prose only) and nine group-theoretic files (`GroupTheory/Complement`, `CosetCover`, `Schreier`, `SchurZassenhaus`, `Transfer`, and others), and no occurrence of *transversal matroid*; plus a reading of every declaration in `Combinatorics/Matroid/Constructions.lean`, which defines exactly `emptyOn`, `loopyOn`, `freeOn` and `uniqueBaseOn`.
- **Isabelle/AFP**: the combinatorics topic index, and searches on *transversal matroid*, *transversal*, *matroid* and *marriage*. Found: *Matroids* (Keinholz, 2018) [21], *Hall’s Marriage Theorem* (Jiang and Nipkow, 2010) [20], and the ITP 2025 matroid/greedoid algorithm work [23]. No transversal-matroid entry located.
- **Mizar**, **Coq/mathcomp**, **HOL Light**: searches on the same terms. Neighbouring finite-set theorems were located in Coq [22]; no transversal-matroid formalization was located in any of the three. This is the weakest leg of the audit: it rests on public search rather than on reading

each library’s index, and I would treat a pointer here as likely to succeed before one aimed at Mathlib or the AFP.

What would refute it. A single locator—library, entry name, file, date—for a formalization of the Transversal Matroid Theorem, or for any Mathlib matroid built from combinatorial data predating revision `84fdd0bf`, refutes the corresponding claim outright. Nothing else in this paper depends on either claim: the Lean development, the axiom profile, the tight-set lattice and Theorem 1 stand or fall on the kernel, not on priority. I would be glad to be sent such a locator, and would record it.

11 Conclusion

I formalized the Transversal Matroid Theorem for finite presentations in Lean 4 over Mathlib, and packaged it as a matroid object, `transversalMatroid A`, whose independent finite sets are identified exactly by `transversalMatroid_indep`. To my knowledge—against the searches dated and itemised in Section 10, and subject to their stated limits—this is the first matroid in Mathlib constructed from combinatorial data rather than from a trivial or dual construction.

I proved augmentation—the one axiom the definition does not give away—by Hall’s obstruction method rather than by augmenting paths, and organized that proof into three independent layers: a matching interface over Mathlib’s finite Hall theorem, a tight-set layer with its own algebra, and a counting layer that converts a cover of tight sets into a single cardinal contradiction. Every step is a separately named, separately reusable lemma.

The resulting object integrates with Mathlib directly. It is produced through `IndepMatroid.ofFinset`, so the library’s rank, closure, basis, duality and minor theory applies to it without further work, and the ground set is taken universal, so that elements outside every presenting set are loops and the characterization is exact. That integration is exercised rather than asserted: Corollary 1 identifies Mathlib’s own `Matroid.closure`, on independent sets, with a single containment test against `maxTight`.

The architecture then paid for itself. Tight sets are closed under intersection as well as union, hence form a lattice; every partial transversal has a maximum tight subset; and one containment test against its neighbourhood decides every insertion (Theorem 1). The obstruction is not an incident of a failed search but a canonical invariant of the pair (presentation, independent set).

Future work, in the order I would attempt it: extract an executable procedure returning either an augmentation or the tight certificate, and connect it to the verified matching algorithms of the Isabelle literature [23]; carry Corollary 1 on to the rank, where the value should be Ore’s defect formula $r(X) = \min_{S \subseteq X} (|X \setminus S| + |N(S)|)$ [13]—verified exhaustively on the running presentation by `check_canonical_obstruction.sage`, and *not* formalized, which is exactly why it is future work rather than a result; lift the construction to infinite indexed families, following Mirsky and Perfect and Mathlib’s own compactness route for Hall; and promote the tight-set layer to a standalone Mathlib contribution, since it is about finite set systems and mentions no matroid at all.

12 Reflections

Formalizing a mid-1960s theorem is not, by itself, interesting. What the exercise taught is, and these seven observations are the residue.

1. A theorem is not the same object as the reason it is true. The Transversal Matroid Theorem asserts that something *can* always be done: a larger transversal always donates. The proof,

however, proceeds entirely by studying what *cannot* be done. Formalizing forced this asymmetry into the open, because a kernel will not accept “one sees that some element must work”; it demands a construction, and the only construction available is of the obstacle. The negative object turned out to be the richer one—it has a lattice, a maximum, and a decision procedure attached—while the positive statement has none of that. This is worth stating as a working heuristic: when a theorem asserts that some choice exists, the formalizable content usually lives in the classification of the reasons it might not.

2. Formal libraries are rich in theory and poor in examples. Mathlib can compute the dual, the minors, the rank and the closure of a matroid, and in revision 84fdd0bf it had essentially no matroid to apply any of that to. The imbalance is structural rather than accidental: theory is stated once and reused, while each family of examples—graphic, linear, transversal—costs its own construction, and constructions get built last. The same pattern is visible elsewhere in every large library. If one wants to be useful to a formal library rather than merely present in it, building an example that its existing theory can chew on may be worth more than another theorem about objects nobody has instantiated.

3. The layering was not tidiness; it was where the mathematics went. Splitting the proof into a matching layer, a tight-set layer and a counting layer was undertaken for readability. It turned out to be the substantive act. Once the tight sets had their own layer, asking whether they are closed under intersection was a natural question with a one-conjunct answer, and Theorem 1 followed. Had the augmentation proof been written as one argument—as it is in the textbooks, and as it would have been in a first Lean attempt—the lattice would have stayed invisible, because there would have been no object to ask the question about. Proof engineering and mathematical discovery were, here, the same activity.

4. A certificate is worth more than a search. An augmenting-path algorithm answers “can e be inserted?” by running, and answers again by running again. The tight set answers once and is then reusable for every subsequent question about the same I . *Theorem of the Day*’s own caveat on this theorem—that greedy selection “leaves open the question of how we know our partial transversals are valid unless we find a matching at the same time”—is precisely a complaint about search, and the canonical obstruction is the reply. That a refusal should come with a reason attached is a design principle that generalizes well past matroids.

5. Classical does not mean available. Every mathematical ingredient here is decades old: Hall 1935, the theorem itself 1965–67, the submodularity of the neighbourhood function older than either. Yet no proof assistant had the statement, and the reason is not difficulty. It is that classical results are assembled from prerequisites that libraries acquire in a different order than textbooks do, so the marginal cost of a theorem in a library has little to do with its marginal cost on paper. This cuts both ways: it explains why some celebrated theorems remain unformalized, and it means a modest classical result can still be a genuine contribution to a library.

6. The right definition costs less than the clever proof. Defining `IsPartialTransversal` as the nonemptiness of Mathlib’s `hallMatchingsOn`, and not as a bespoke predicate, was the single decision that made the development short. It bought the reverse Hall direction outright, and it made every later lemma a statement about cardinalities rather than about functions. The corresponding temptation—to define a convenient predicate and then prove it equivalent to the library’s—would

have added a translation layer to every proof. Where a library already speaks about your object, it is almost always cheaper to speak its dialect than to introduce your own.

7. What the assistant changed, and what it did not. This development was written in close interaction with an AI assistant, and it is worth being precise about the effect. It compressed the mechanical distance between an idea and a checked proof, so more architectural variants could be tried in an afternoon than would otherwise fit in a week—and it was that cheap iteration, not any single insight, that surfaced the intersection half of Proposition 1. What it did not change is the standard of truth. The kernel accepted or refused, the axiom report is machine-generated, and the scope statements in Section 9 are claims a human is answerable for. A faster way to try things does not lower the bar; if anything, it makes stating the bar explicitly more important.

13 Open ends

Four questions the work raises but does not settle.

1. *From witness to algorithm.* The tight witness already explains a refusal. Can the existence proof be turned into a verified procedure returning either a larger transversal or a checkable saturated-neighbourhood certificate—and does the maximum tight set, obtained from a Dulmage–Mendelsohn decomposition, make that procedure polynomial with the certificate for free?
2. *Which finiteness is really needed.* The proof uses finiteness in three separate places: to choose witnesses, to take a finite union, and to do cardinal arithmetic. Which of the three survives to infinite presentations, and is Mathlib’s compactness argument for Hall enough to carry the rest?
3. *Does the tight-set layer belong in Mathlib?* `tight_union_and_inter` and `tight_biUnion` say nothing about matroids. They are lemmas about a finite set system and the equality case of Hall’s condition. That is a Mathlib-shaped statement looking for a home.
4. *One obstruction, three faces.* The lattice of tight sets is the same phenomenon as the Dulmage–Mendelsohn decomposition of a bipartite graph and as the canonical maximal violator in submodular optimization. Is there one Lean notion—a submodular function and its equality lattice—that subsumes all three, with the transversal case an instance? That is the bead I would most like to carry forward.

Reproducing

```
lake build Theorems.TransversalMatroid      # the construction
lake build Theorems.TransversalObstruction  # the canonical obstruction
lake env lean formalizations/transversal-matroid/lean/
        Theorems/AxiomReport.lean          # the trust report
python formalizations/transversal-matroid/sage/check_canonical_obstruction.sage
```

Lean 4, toolchain `leanprover/lean4:v4.32.0`, over Mathlib at revision `84fdd0bf` (2026-07-15):

<https://github.com/leanprover-community/mathlib4>
pinned at `84fdd0bfb8e85237f54b9617cca79c6198d40a75`

The two Mathlib modules this development stands on, at that revision, are `Combinatorics/Hall/Basic.lean` and `Combinatorics/Matroid/IndepAxioms.lean`; the constructions module whose contents Section 10 reports is `Combinatorics/Matroid/Constructions.lean`. Every absence claim in this paper is checkable against those exact URLs, and remains checkable after Mathlib moves on. To reproduce elsewhere, point `lakefile.lean` at a compiled Mathlib with the same toolchain; the local path used here is recorded in `MATHLIB_ENVIRONMENT.md`.

The Lean sources, the generated axiom report, the exhaustive SageMath cross-check and the deterministic figure generator are archived with the other formalizations of this series [29, 30, 31] at <https://github.com/karlesmarin/godsil-gutman-lean>. Both Lean files are `sorry`-free, and `#print axioms` on the six theorems listed under Table 1 reports only `propext`, `Classical.choice`, `Quot.sound`. The source-level verification record is `VERIFICATION.md`; the running example of every figure is re-derived exhaustively—all 56 partial transversals, all 180 insertion decisions—by `check_canonical_obstruction.sage` before any figure is drawn.

References

- [1] P. Hall, On representatives of subsets, *J. London Math. Soc.* **10** (1935), 26–30. <https://doi.org/10.1112/jlms/s1-10.37.26>.
- [2] J. Edmonds, D. R. Fulkerson, Transversals and matroid partition, *J. Res. Nat. Bur. Standards Sect. B* **69B** (1965), 147–153.
- [3] L. Mirsky, H. Perfect, Applications of the notion of independence to problems of combinatorial analysis, *J. Combin. Theory* **2** (1967), 327–357.
- [4] H. Perfect, Applications of Menger’s graph theorem, *J. Math. Anal. Appl.* **22** (1968), 96–111. [https://doi.org/10.1016/0022-247X\(68\)90163-7](https://doi.org/10.1016/0022-247X(68)90163-7).
- [5] R. Whitty, The Transversal Matroid Theorem, *Theorem of the Day*, <https://www.theoremoftheday.org/CombinatorialTheory/Transversal/TotDTransversal.pdf> (attribution: “proved in the mid-1960s by Jack Edmonds and Delbert Ray Fulkerson in the USA and, independently (and also in an infinite version) by Leon Mirsky and Hazel Perfect at Sheffield University in the UK”).
- [6] R. Rado, A theorem on independence relations, *Quart. J. Math. Oxford Ser.* **13** (1942), 83–89 (Hall’s criterion generalised to matroids).
- [7] H. Perfect, *Studies in Transversal Theory with Particular Reference to Independence Structures and Graphs*, Ph.D. thesis, University of London, 1969.
- [8] H. Perfect, L. Mirsky, Spectral properties of doubly-stochastic matrices, *Monatsh. Math.* **69** (1965), 35–57. <https://doi.org/10.1007/BF01313442> (the source of the Perfect–Mirsky conjecture).
- [9] J. Oxley, The contributions of Dominic Welsh to matroid theory, <https://www.math.lsu.edu/~oxley/dominic.pdf> (on the Oxford and Sheffield schools, and on Edmonds’s 1965 National Bureau of Standards conference).
- [10] H. Whitney, On the abstract properties of linear dependence, *Amer. J. Math.* **57** (1935), 509–533 (the paper that names the matroid, published the same year as Hall’s).
- [11] G. Farr, J. Oxley, The contributions of W. T. Tutte to matroid theory, https://www.math.lsu.edu/~oxley/Tutte_farrox5.pdf (on the dormancy of matroid theory after 1935 and its revival with Tutte’s papers of 1958–59).

- [12] V. Bryant, H. Perfect, *Independence Theory in Combinatorics: An Introductory Account with Applications to Graphs and Transversals*, Chapman and Hall, 1980.
- [13] O. Ore, Graphs and matching theorems, *Duke Math. J.* **22** (1955), 625–639 (the defect form of Hall’s theorem).
- [14] A. L. Dulmage, N. S. Mendelsohn, Coverings of bipartite graphs, *Canad. J. Math.* **10** (1958), 517–534.
- [15] L. Lovász, M. D. Plummer, *Matching Theory*, North-Holland, 1986.
- [16] D. J. A. Welsh, *Matroid Theory*, Academic Press, 1976.
- [17] J. G. Oxley, *Matroid Theory*, 2nd ed., Oxford University Press, 2011.
- [18] A. W. Ingleton, Transversal matroids and related structures, in *Higher Combinatorics*, NATO ASI Series C **31**, Reidel, 1977, 117–131.
- [19] A. Gusakov, B. Mehta, K. A. Miller, Formalizing Hall’s marriage theorem in Lean, 2021. <https://arxiv.org/abs/2101.00127> (the Mathlib development used here: `Mathlib.Combinatorics.Hall.Basic`).
- [20] D. Jiang, T. Nipkow, Hall’s Marriage Theorem, *Archive of Formal Proofs* (2010). <https://isa-afp.org/entries/Marriage.html>.
- [21] J. Keinholtz, Matroids, *Archive of Formal Proofs* (2018). <https://isa-afp.org/entries/Matroids.html>.
- [22] A. Singh, R. Natarajan, Fully mechanized proofs of Dilworth’s theorem and Mirsky’s theorem, 2017. <https://arxiv.org/abs/1703.06133> (Coq; neighbouring finite-set theorems).
- [23] *A Formal Analysis of Algorithms for Matroids and Greedoids*, *Interactive Theorem Proving (ITP 2025)*, LIPIcs, Schloss Dagstuhl. <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.2> (includes executable bipartite maximum matching).
- [24] *Formalizing the Excluded Minor Characterization of Binary Matroids in the Lean Theorem Prover*, University of Waterloo, 2023. <https://uwspace.uwaterloo.ca/items/d35d17be-485d-4f60-8696-4dce7ae907bb>.
- [25] *Composition Direction of Seymour’s Theorem for Regular Matroids — Formally Verified*, 2025. <https://arxiv.org/abs/2509.20539>.
- [26] The mathlib Community, The Lean mathematical library, in *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381. <https://doi.org/10.1145/3372885.3373824>. Repository: <https://github.com/leanprover-community/mathlib4>; this work is checked against revision 84fdd0bfb8e85237f54b9617cca79c6198d40a75 (2026-07-15).
- [27] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37. Repository: <https://github.com/leanprover-community/lean4>.
- [28] The Sage Developers, *SageMath, the Sage Mathematics Software System*. <https://www.sagemath.org>.
- [29] C. Marín, *When Separation Fails, a Canon Appears: one counting identity behind separation, synchronization and symmetry-breaking in permutation groups, formalized in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.21419285 (the previous *Theorem of the Day* bead; same sourcing method, different cluster).
- [30] C. Marín, *Random Signs into Matchings: A Godsil–Gutman Identity, Formalized in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.20517350 (matchings of a bipartite graph, counted rather than chosen).

- [31] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton's Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026. doi:10.5281/zenodo.20693064 (log-concavity, the classical ancestor of matroid log-concavity).